

Rapport projet BDI

Lien github de mon projet : https://github.com/tryckyy/BDI_project.git

Conception d'un Simulateur Multi-Agent pour la Gestion du Trafic

Rapport de projet par Hurst Lilian

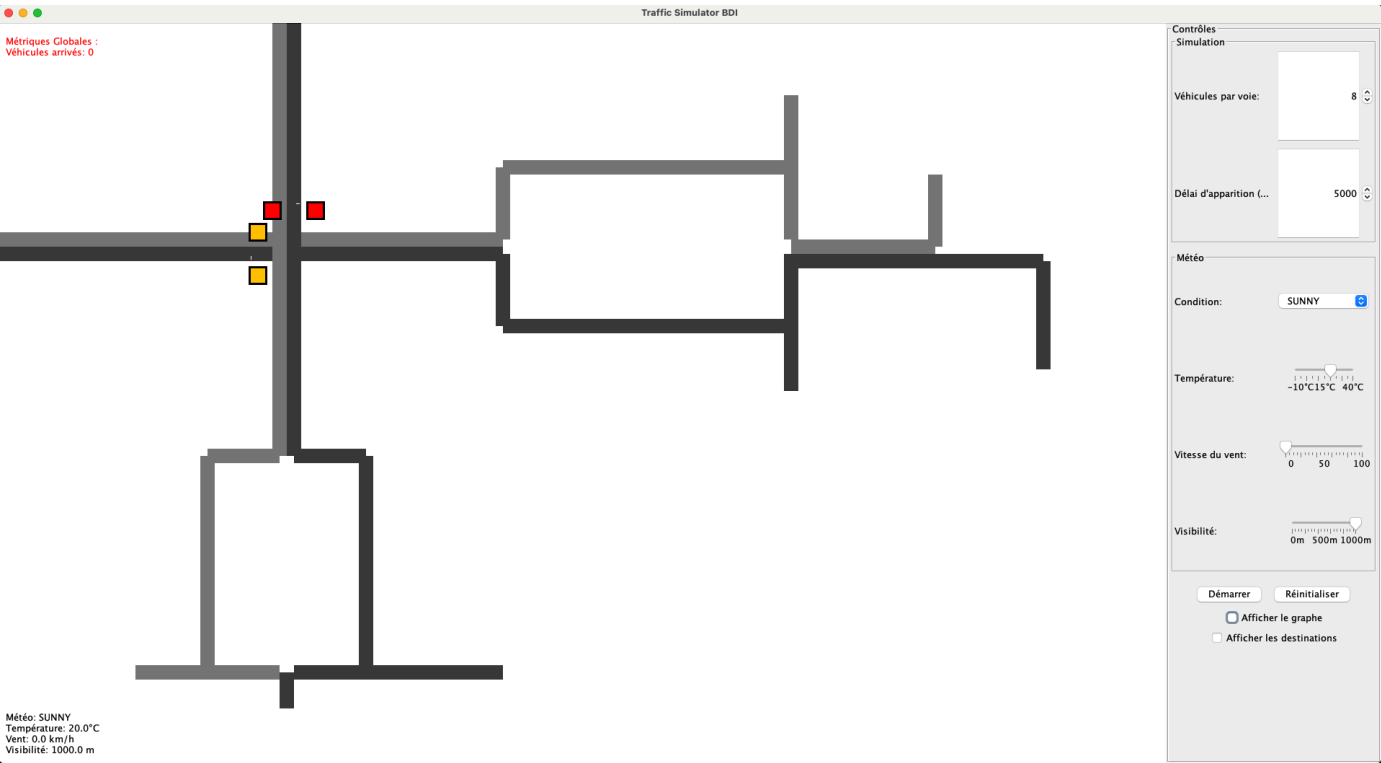
Objectif du projet

L'objectif de ce projet est de concevoir un simulateur multi-agent appliqué à la gestion du trafic urbain. Les étudiants devront modéliser des agents capables de prendre des décisions autonomes pour simuler le trafic et optimiser la circulation. Ce projet permet de mettre en application les techniques de planifications vues en cours dans un cadre concret et dynamique.

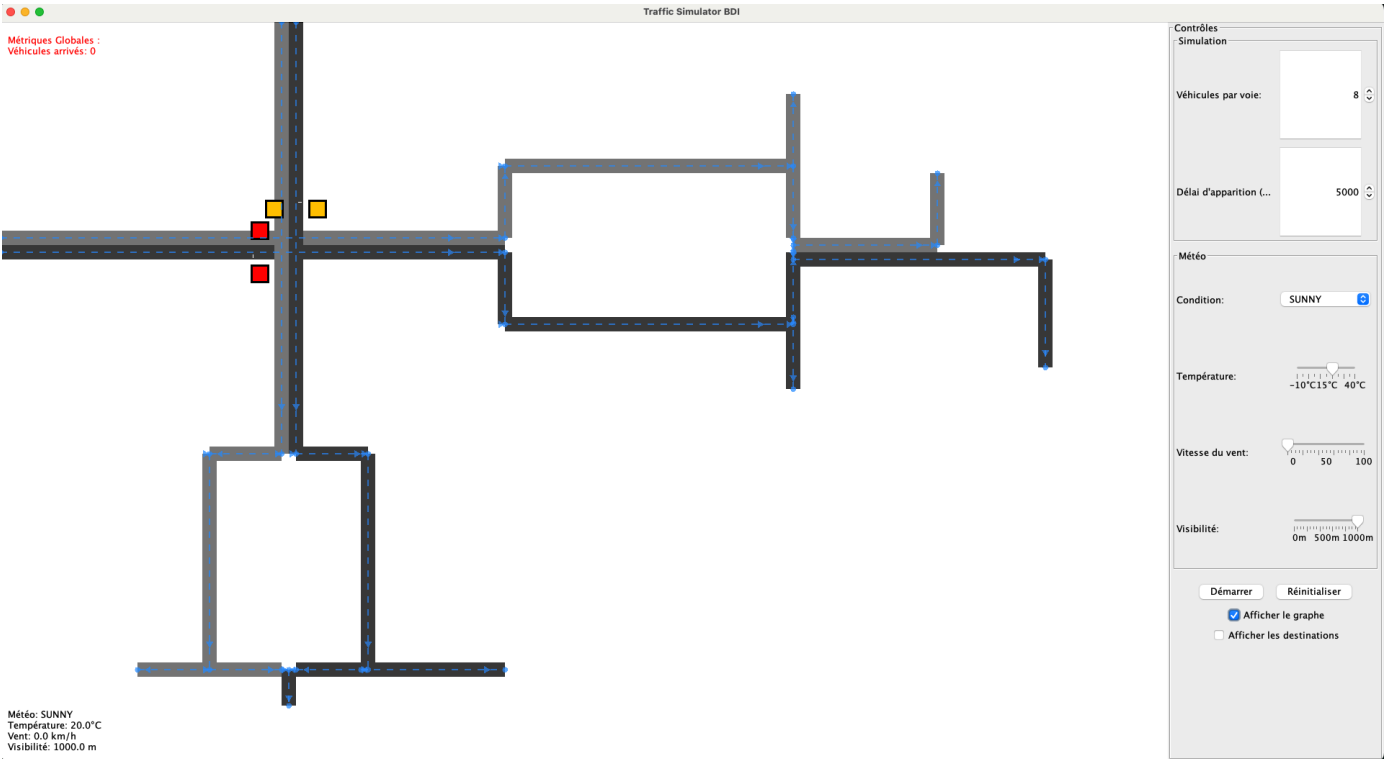
Architecture du projet

L'architecture du projet suit le modèle MVC (Modèle-Vue-Contrôleur), une structure classique pour les applications interactives, qui sépare clairement les responsabilités entre les composants.

Interface graphique



Affichage du graphe Dijkstra



Affichage des chemins possibles et des destinations

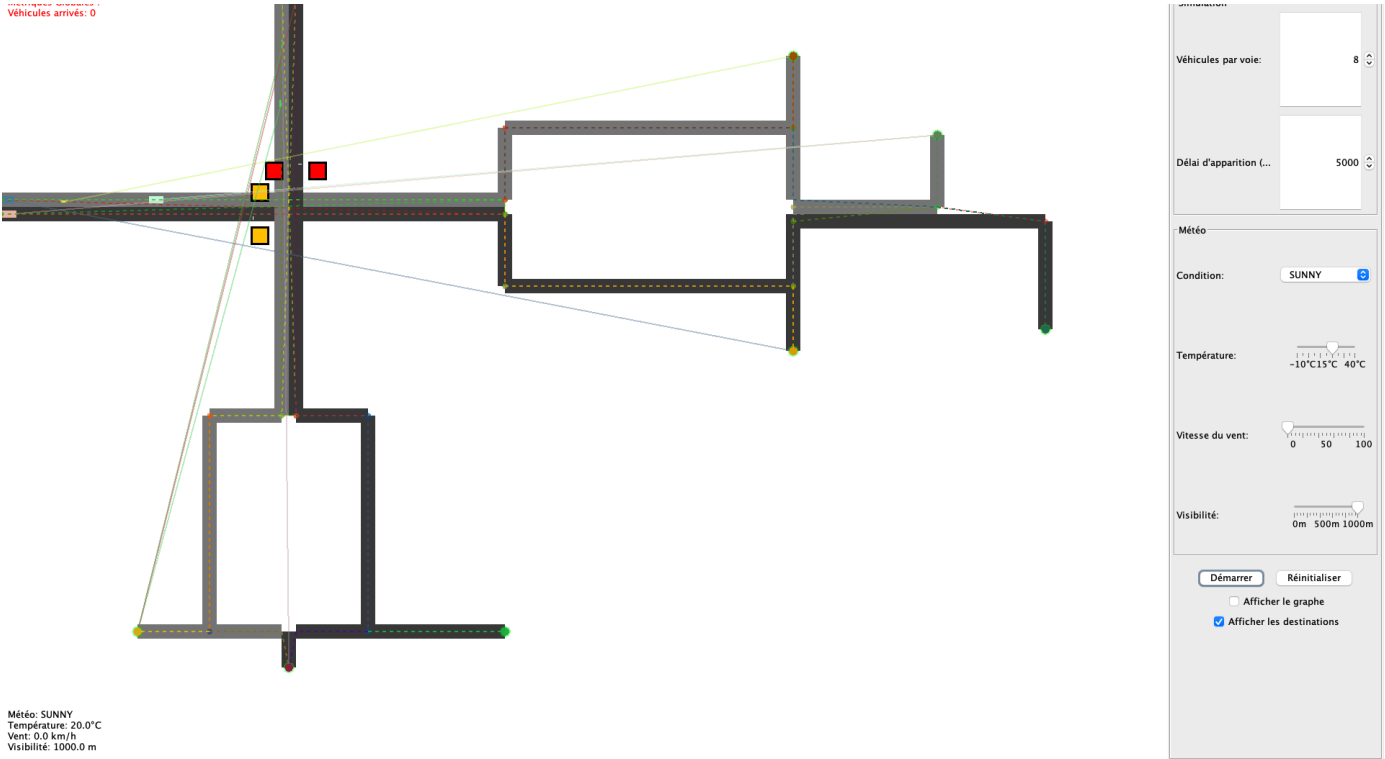
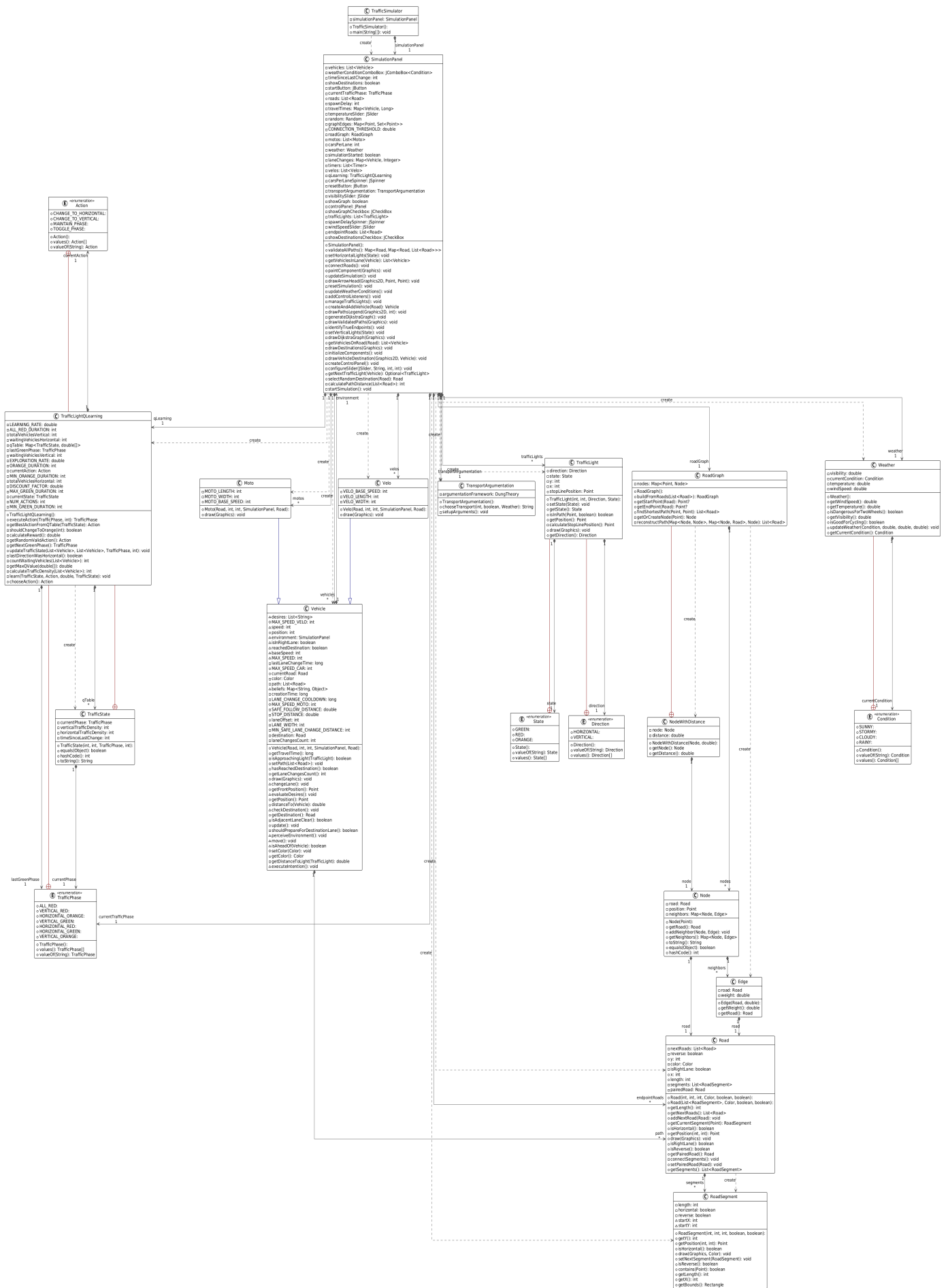


Diagramme de classe



1 Modèle du SMA

Types d'agents :

- Agents Véhicules : Entités autonomes avec une logique BDI.
- Contrôleurs de Feux : Entités non-agents gérées par la simulation.

Rôles et objectifs :

- **Agents Véhicules :**

- Rôle : Naviguer de leur point de départ à leur destination.
- Objectifs : Éviter les collisions, respecter les feux, optimiser le temps de trajet.

- **Feux de Circulation :**

- Rôle : Contrôler les intersections via des phases temporelles.
- Objectif : Minimiser les embouteillages.

- **Agents Moto :**

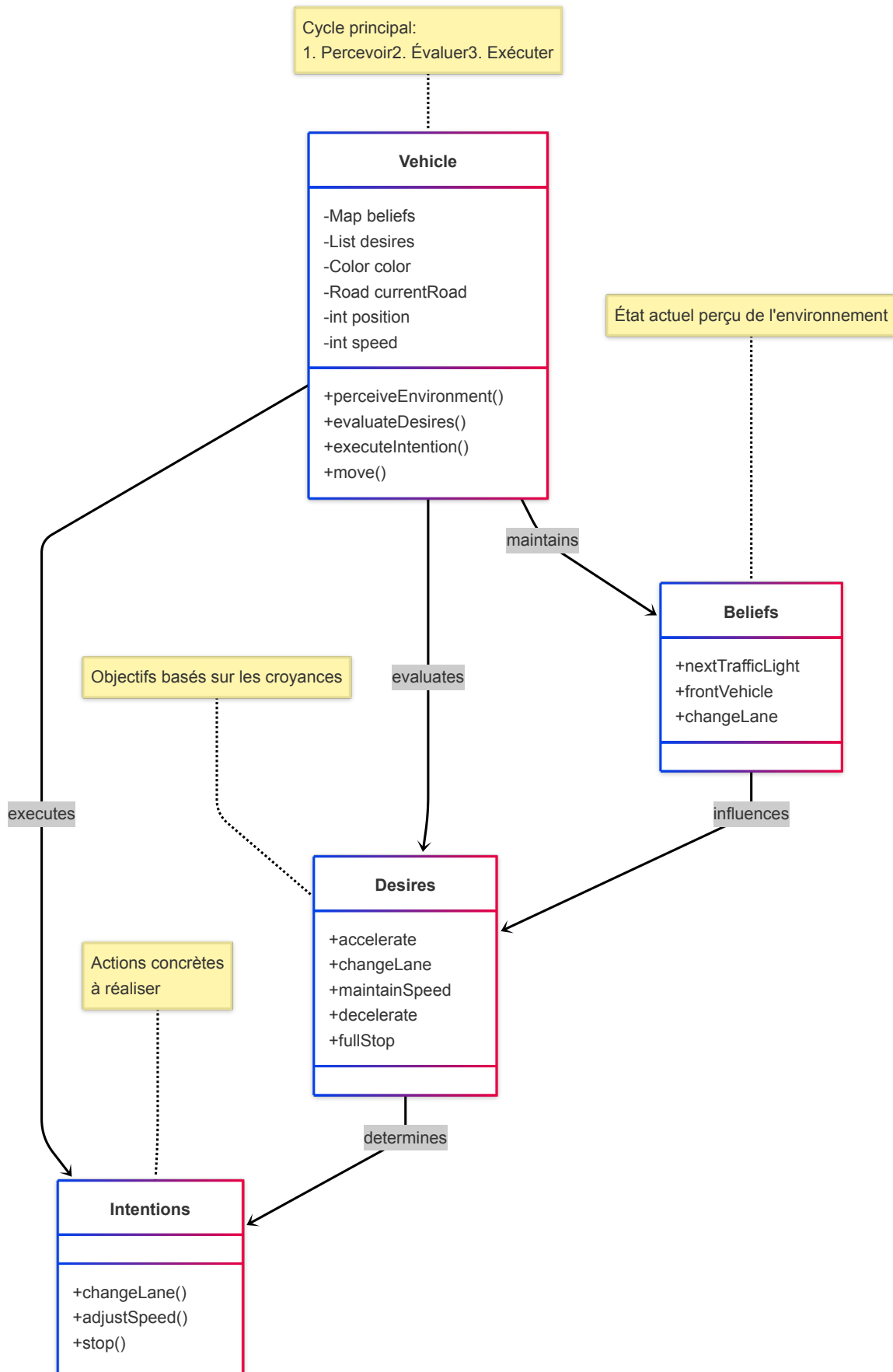
- Rôle : Naviguer de leur point de départ à leur destination.
- Objectifs : Éviter les collisions, respecter les feux, optimiser le temps de trajet.
- Caractéristique : vas moins vite que les voitures et est plus petit que les voitures

- **Agents Velo :**

- Rôle : Naviguer de leur point de départ à leur destination.
- Objectifs : Éviter les collisions, respecter les feux, optimiser le temps de trajet.
- Caractéristique : vas moins vite que les voitures et les motos et est plus petit que les voitures et les motos.

2. Modélisation des agents et des interactions

2.1 Architecture BDI des véhicules



Le système utilise une architecture BDI (Beliefs-Desires-Intentions) pour modéliser le comportement des véhicules.

L'architecture BDI est implémentée dans la classe **Vehicle**, avec des structures dédiées pour chaque composant.

- **Beliefs** : Représentent la perception de l'environnement par le véhicule

Représentation : Map<String, Object> beliefs

Mécanisme de mise à jour : méthode *perceiveEnvironment*

- Perception du véhicule devant grâce à la méthode *frontVehicle*.
- Perception du prochain feu de circulation grâce à la méthode *nextTrafficLight*
- Déterminer si il faut changer de voie pour la prochaine route grâce à la méthode *shouldPrepareForDestinationLane*

-
- **Desires** : Objectifs que le véhicule cherche à atteindre

Représentation : List <String> desires

Processus de génération : méthode *evaluateDesires*

- Tout d'abord, les anciens desires sont **effacés**
- Si il n'y a pas de **beliefs** de changement de voie, j'**accélère**, sinon j'ajoute le **desire** de changer de voie.
- Si un véhicule se trouve en face, je récupère la distance, si il est plus proche que la **SAFE_FOLLOW_DISTANCE**, j'ajoute le **desire** de ralentir. Si il est plus proche que la **STOP_DISTANCE**, j'ajoute le **desire** de s'**arrêter**.
- Si une voiture s'approche d'un feu de route, j'ajoute le **desire** de **ralentir**. Si le feu est orange ou rouge, j'ajoute le desire de s'**arrêter**. Si il est vert je **maintiens** ma vitesse.

-
- **Intentions** : Actions concrètes sélectionnées parmi les désirs

Représentation : Exécution sélective des désirs

Processus de génération : méthode *executeIntentions*

- En fonction des desires qui sont stocker, les différentes méthodes contenant la logique seront appelés.
- Pour le desire **changeLane**, la méthode *changeLane* est appelé, cette méthode vérifie si la voie adjacente est libre et si oui elle déplace la voiture sur la voie correspondante.
- Pour le desire **maintainSpeed**, la vitesse est maintenu à la valeur de la constante **baseSpeed**.
- Pour le desire **accelerate**, la voiture accélère en fonction de la voie ou elle situé, elle ira plus vite sur la voie de gauche et moins vite sur la voie de droite. Cela évite les dépassements par la droite et reproduit un comportement réaliste.
- Pour le desire **fullStop**, la vitesse est mise à 0.

2.2 Interactions entre agents

Les interactions principales sont

- Véhicule → Environnement
 - Perception des feux de circulation via *getNextTrafficLight*
 - Détection des autres véhicules via *getVehiclesInLane*
- Véhicule → Véhicule
 - Respect des distances de sécurité
 - Communication implicite via le comportement (ralentissement, changement de voie)
- Véhicule → Infrastructure
 - Réaction aux feux de circulation
 - Utilisation du réseau routier via le graph des routes (Dijkstra)

3 Choix des algorithmes de planification utilisés

3.1 Algorithme de Dijkstra

Choix justifié

- Nécessité de trouver les chemins les plus courts dans un graphe pondéré
- Précis et garanti de trouver la solution optimale
- Adapté aux graphes routiers avec pondérations variables

Optimisation spécifiques

- Pénalités de changement de voie (+10 de coût)
- Simplification post-traitement pour éliminer les doublons
- Cache des résultats pour les paires source-destination fréquentes

Fonctionnement

- **Modélisation du graphe**
 - Nœuds : Points de départ/arrivée des routes et points intermédiaires pour les changements de voie.
 - Arêtes : Routes connectées, avec des poids basés sur la longueur et des pénalités pour les changements de voie.

- **Processus**

- Initialisation des distances (infini sauf pour le nœud de départ).
- Sélection itérative du nœud le plus proche.
- Mise à jour des distances des voisins.
- Reconstruction du chemin à partir des nœuds précédents.

3.2 Algorithme Q-Learning pour la Gestion des Feux de Circulation

Choix justifié

L'algorithme Q-Learning a été choisi pour gérer les feux de circulation en raison de sa capacité à apprendre des politiques optimales dans des environnements stochastiques et dynamiques. Contrairement aux approches prédéfinies (comme les temporisations fixes), le Q-Learning permet une adaptation en temps réel aux conditions de trafic, ce qui améliore l'efficacité globale du système.

Fonctionnement

- **États** (TrafficState) :
 - Représentent la densité du trafic (horizontale et verticale), la phase actuelle des feux, et le temps écoulé depuis le dernier changement.
- **Actions** (Action):
 - **MAINTAIN_PHASE**: Garder la phase actuelle.
 - **CHANGE_TO_HORIZONTAL/CHANGE_TO_VERTICAL** : Changer de direction.
 - **TOGGLE_PHASE** : Alternner entre les directions.
- **Récompenses**:

Calculées en fonction:

 - Du nombre de véhicules à l'arrêt (pénalité).
 - Du flux de véhicules (récompense).
 - Des changements de phase trop fréquents (pénalité).
 - Du respect des durées minimales/maximales des phases.

- **Mise à jour de la Q-table**

- Utilisation de la formule :

$$Q[s, a] \leftarrow Q[s, a] + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- **Politique ϵ -greedy**

Alterne entre exploration (actions aléatoires) et exploitation (meilleure action connue) pour équilibrer apprentissage et performance.

- **Avantages**

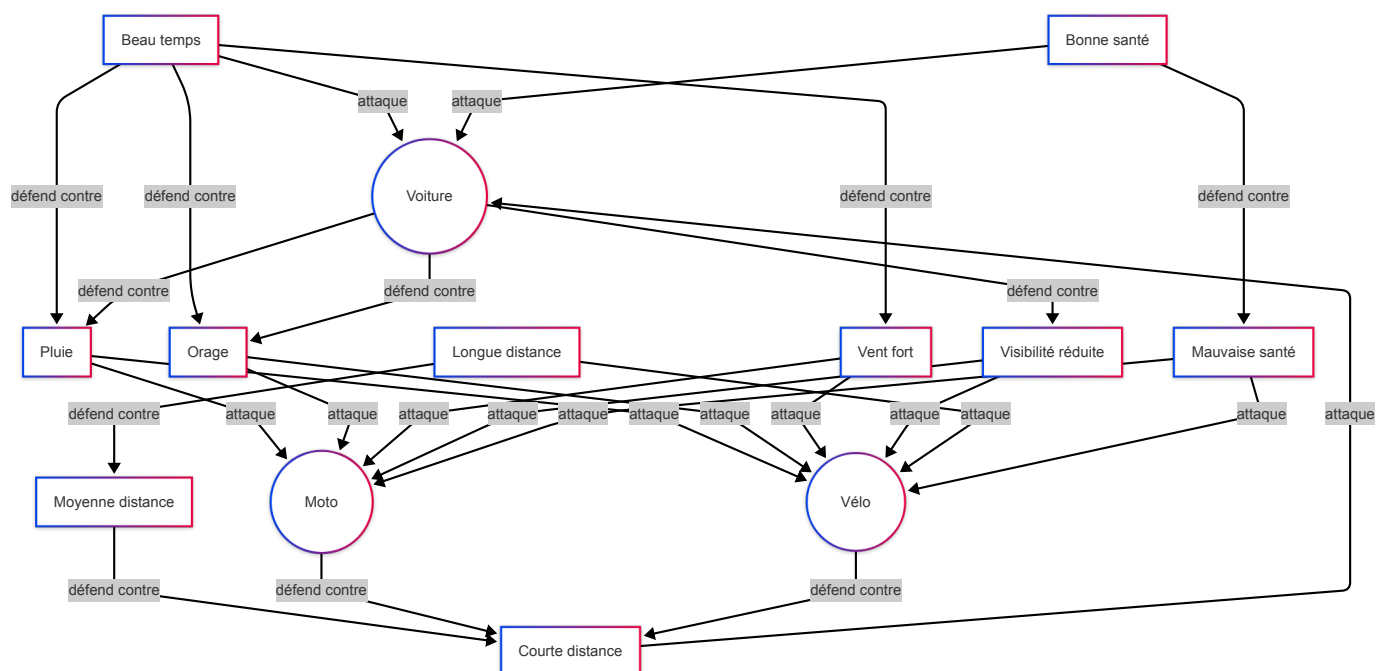
Adaptabilité aux variations de trafic.

Optimisation des temps d'attente et du flux global.

3.3 Argumentation

La classe **TransportArgumentation** implémente un système d'argumentation basé sur le framework de Dung pour aider à la décision du choix de moyen de transport. Elle utilise la bibliothèque Tweety pour gérer le raisonnement argumentatif.

Graphique d'argumentation



Structure de l'argumentation

Arguments :

- Moyen de transports (voiture, vélo, moto)
- Conditions météorologiques
- Distance
- Etat de santé

Relations d'attaques

- Météo → Transport (pluie attaque moto / vélo)
- Distance → Transport (longue distance attaque vélo)
- Santé → Transport (mauvaise santé attaque vélo / moto)

Initialisation (*setupArguments*)

- Configuration des arguments
- Définitions des relations d'attaque
- Création du réseau d'argumentation

Prise de décision (*chooseTransport*)

- **distance** : distance à parcourir
- **bonneSante** : état de santé
- **weather** : Conditions météorologiques

Règles de décision principal

- Conditions météorologiques défavorables → Voiture
- Bonne santé + Courte distance → Vélo
- Distance moyenne + Conditions acceptables → Moto

Choix de l'extension

Une extension préférée est un ensemble maximal d'arguments.

Cela garanti la considération du plus grand nombre possible d'arguments compatibles et résout les situation contradictoires.

4 Analyse des performances et des résultats obtenus

Vehicule par voies (par type)	Véhicules arrivés	temps moyen	changement moyen
1	12/12	64,3	0,4
2	24/24	77,6	0,6
3	36/36	130,5	0,9

Analyse

Le nombre de véhicules total généré correspond au nombre de véhicules par voie multiplié par le nombre de types de véhicules. Dans ce cas si, j'ai 3 types de véhicules (vélo, moto, voiture).

Les véhicules arrivent tous à leur destination respective sans collisions. Le temps augmente principalement à cause de l'attente au feu. Je peux faire le test qu'avec un maximum de 6 voitures par voies car avec plus que ce nombre les voitures apparaissent l'une sur l'autre car la route n'est pas assez longue pour ce nombre. Les voitures respectent le code de la route et arrive à destination via différents chemins possibles.

Conclusion

Ce simulateur de trafic BDI démontre une approche efficace pour modéliser des comportements complexes dans un environnement dynamique. La combinaison des architectures BDI, des algorithmes de planification de trajectoire et des techniques d'apprentissage par renforcement permet de créer un système réaliste et adaptable. Les résultats obtenus montrent la viabilité de cette approche pour des applications plus larges en simulation de trafic et en gestion intelligente des transports.